RB2301 CA1 Report

No.	Name	Student ID
1.	Maxwell Timothy Ebora Guico	A0323869N
2.	**Suda Jesmitha Reddy**	**A0321806L**
3.	Nguyen Linh Nhien	A0325663B

Introduction

The CA1 problem is to move a mobile robot through a field of coke cans without using a pre-built map. The robot receives a two-dimensional LiDAR scan and publishes velocity commands through ROS 2. It must decide from the current scan whether the path ahead is clear, whether a visible gap is wide enough, or whether it should move toward a safer side.

This problem is harder than simply looking for the largest distance reading. The LiDAR measures from one point on the robot, but the robot has a real width. Two cans can appear far apart when their separation is measured diagonally, while the space across the robot's planned route is still too narrow. A can that is closer to the robot can also block the route to a gap that looks open farther away.

Overview of the node

ObstacleAvoidanceNode is the main ROS 2 node for obstacle avoidance. It subscribes to /scan for LiDAR data, publishes motion commands to /cmd_vel, and runs the selected algorithm through a 20 Hz timer callback. The algo parameter selects either "nhien" or "max". The node stores every second LiDAR range reading in a NumPy array, while move_2D(x, y, turn) clips and publishes forward or backward velocity (x, ±0.4 m/s), lateral velocity (y, ±0.4 m/s), and rotational velocity (turn, ±0.8 rad/s).

Chosen Methodology, algo = max

Checking the Forward Corridor

The LiDAR represents the visible surface of each can using scan points separated by 2 degrees. To determine whether straight-line motion is possible, the controller defines a rectangular corridor extending 0.6 m in front of the robot and 0.12 m on either side of its centreline, giving a total width of 0.24 m. Any detected can point within this region causes the forward path to be classified as blocked. If the corridor is clear, the robot travels forward at 0.30 m/s. Following an avoidance manoeuvre, eight consecutive clear scans, corresponding to 0.40 s, are required before straight motion resumes.

Grouping Cans and Finding Gaps

Adjacent LiDAR returns are grouped as belonging to the same can when the detected positions are separated by no more than 0.12 m. For each pair of neighbouring cans, the controller identifies the right inner edge of one can and the left inner edge of the next. The candidate target is placed at the midpoint between these two inner edges.

Only candidate gap midpoints within the 170-degree forward field of view are considered, spanning 85 degrees to the left and 85 degrees to the right of the robot. An opening is accepted only if its usable width along the intended direction of travel is at least 0.33 m and the route to it is not obstructed by a closer can.

Choosing a Motion

The controller gives priority to a safe opening located close to the robot’s forward direction. If no such opening is available, the current preferred-side configuration causes right-side gaps to be evaluated first. The robot approaches the selected gap midpoint at 0.18 m/s, this target lies halfway between the inner edges of the two cans. When no acceptable opening exists on the preferred side, the fallback behaviour compares the available clearance on the two sides.

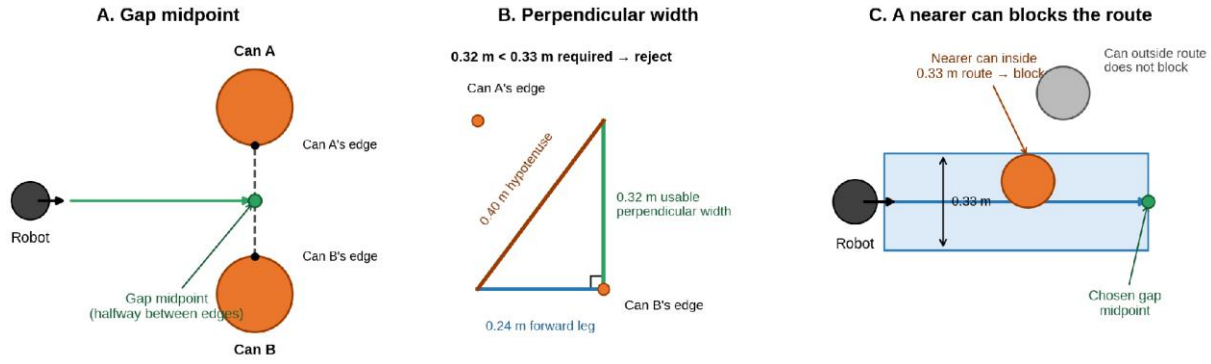
Fallback and Close-Range Safety

When no valid gap can be used, the controller compares the median LiDAR range on the left and right and chooses the side with greater clearance. Before issuing a lateral command of 0.25 m/s, it verifies that a 0.30 m path beside the robot is free. If the chosen side is obstructed while the opposite side remains clear, the controller changes direction and maintains that choice for 12 cycles. If both sides are blocked, or an obstacle is detected within 0.20 m in front, the robot reverses at 0.08 m/s.

The Math Behind the Algorithm

Each LiDAR sample gives a distance r and an angle theta. The controller uses these polar measurements to calculate the position of the detected point relative to the robot, where positive x represents the forward direction and positive y represents the left direction.

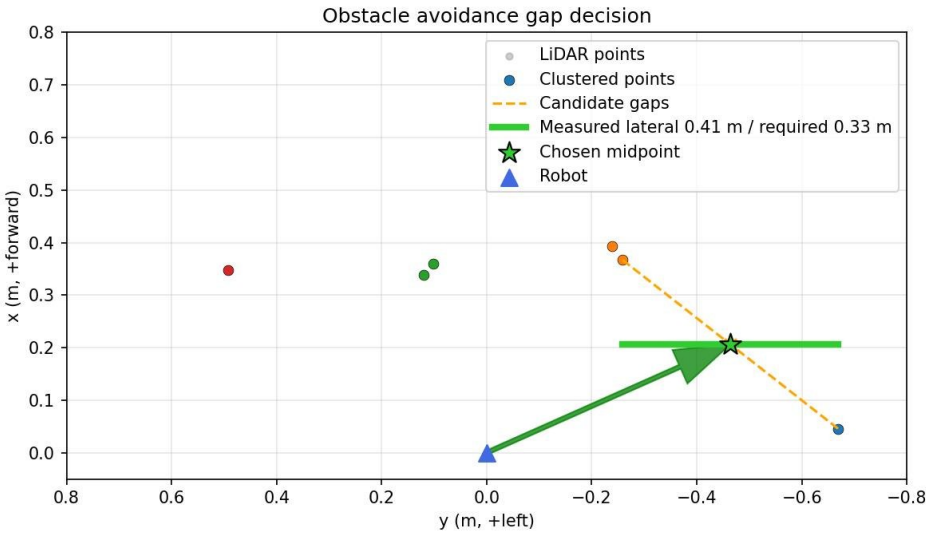
$$x = r\cos(\theta), \qquad y = r\sin(\theta)$$



Perpendicular Gap Width

Let g denote the vector joining the two inner can edges and let d be a unit vector that directed from the robot to the gap midpoint. The clearance is the component of g that is perpendicular to d. In two dimensions, the controller calculates this perpendicular width using the following cross-product expression shown below.

$$w_{\perp} = |d_x g_y - d_y g_x|$$



As shown in the figure above, the orange dashed segment represents a candidate opening. The controller projects this gap perpendicular to the intended direction of motion, producing the lateral clearance shown in green. The resulting width is 0.41 m, which is greater than the required 0.33 m. The opening is therefore considered traversable, and the green-star midpoint becomes the next navigation target.

Robot Width and Route Check

$$w_{\min} = 0.23 + 2(0.05) = 0.33 \text{ m}$$

The robot has a physical width of 0.23 m. A safety allowance of 0.05 m is added on both sides, resulting in a minimum required clearance of 0.33 m. After a gap satisfies this width requirement, the controller also verifies that the complete path from the robot to the selected midpoint remains unobstructed.

Code Explanation

Function	Explanation
timer_callback()	Processes the most recent LiDAR scan, determines whether forward travel is possible, selects either a safe gap or a sideways escape action and sends a movement command.
find_gap_targets()	Clusters LiDAR returns belonging to the same obstacle, identifies gaps between neighbouring obstacle, and retains only openings that provide sufficient width and an unobstructed route.
choose_gap()	Selects a suitable traversable gap, prioritising openings near the forward direction. If none is available, it chooses a gap on the preferred side that requires minimal lateral displacement.
get_sidestep_command()	Generates fallback motion when no acceptable gap is available by moving towards the clearer side, switching sides when necessary, and reversing slightly when an obstacle is dangerously close in front.
get_front_scan()	Takes the front half of the LiDAR scan, filters invalid measurements, and converts the remaining ranges into distances into forward and sideways coordinates.
required_clearance_for_direction()	Calculates the empty space needed to travel in each direction, while accounting for the robot's physical width and safety allowance.
move_2D()	Publishes the commanded forward, lateral, and rotational velocities while enforcing the specified speed limits.
sub_scan_callback()	Receives incoming LiDAR data and stores a down-sampled scan for main controller.
__init__()	Initialises the ROS node together with its publishers, subscribers, timers, motion parameters, safety thresholds, evaluation variables, and debugging options.

Empirical Results

Parameters	height	size_div	width	randomise	Edge_*case*_spacing
Value	20	5	22	True	0.2

Seed	Time (simulated s)	Path length (m)	Cans	Success
1	26.28	7.220	46	Yes
2	26.46	7.281	43	Yes
3	24.94	7.161	40	Yes
4	27.48	7.420	44	Yes
5	24.84	7.162	42	Yes
Total	**130.00**	**36.244**	—	**5/5**

Average	**26.00**	**7.249**	—	—

Parameter	Number of runs	Success Rate (%)	Average Time Taken (s)	Average Path Length (m)
algo=nhien	**5**	**100**	**19.86**	**7.5598**
algo=max	**5**	**100**	**26.00**	**7.249**

Comparing the two algorithms, algo = nhien achieved a lower average completion time of 19.86 s, compared to 26.00 s for algo = max. However, algo = max produced a slightly shorter average path length of 7.249 m, compared to 7.5598 m for algo = nhien. Both algorithms achieved a 100% success rate across five runs. The difference in completion time may be partly attributed to the different movement speeds used by the two algorithms, while the difference in path length reflects variations in their obstacle-avoidance and path-selection strategies.

Reflections

CA1 provided several lessons that we can apply to future robotics projects, particularly the importance of iterative testing and accounting for differences between simulation and physical hardware. During development, there were instances where the robot successfully navigated a configuration in simulation but was unable to reproduce the same behaviour consistently on the actual course. This may have resulted from real-world effects such as positional drift, sensor noise, wheel slip, or small variations in obstacle placement that are difficult to represent perfectly in simulation. This taught us not to rely solely on successful simulation results and to allocate sufficient time for repeated hardware testing and parameter tuning.

We also learned the value of modular debugging, where individual components of the algorithm were tested separately before being integrated into the complete system. From a teamwork perspective, having members develop and evaluate different approaches allowed us to compare solutions and identify weaknesses from multiple perspectives. Clear communication was especially important when integrating these ideas into a single working system.

One limitation of our current approach is that the controller primarily reacts to nearby LiDAR measurements without explicitly correcting for positional drift or considering the complete route through the course. As a result, small motion inaccuracies on the physical robot may accumulate, while locally optimal decisions may not necessarily lead to the most effective overall path. In future, incorporating localisation feedback could allow the robot to detect deviations from its expected trajectory and correct its motion accordingly. A more comprehensive planning strategy could also be explored to account for obstacles beyond the robot's immediate surroundings.

Overall, CA1 showed us that developing a successful robotics system requires more than designing a functional algorithm. Robust real-world testing, systematic debugging, effective collaboration, and the ability to adapt when physical behaviour differs from simulation are equally important. These are lessons we intend to carry forward into future robotics projects.

Github Repo: Source Code

AI Declaration

I used GPT-6 Sol to generate code and visualisations for my gap following algorithm. I am responsible for the content and quality of the submitted work.